StudyBuddy Full-Stack System Overview and Architecture

CSL7090 Software and Data Engineering

| V Sujay (B22CS063) |Aiswarya K (B22CS028) |

26 SEPTEMBER 2025

Code Files

Contents

1 Executive Summary

StudyBuddy is a web-based platform designed to connect university students with compatible study partners. Its primary goal is to facilitate collaborative study by matching students based on shared courses and schedules. The system provides end-to-end account management (signup, login, profile editing), private and group chat functionality, and session scheduling, all within a secure, responsive single-page application.
The front-end is built as a React/TypeScript SPA served by an Express.js backend. The backend exposes RESTful APIs and real-time endpoints (using Socket.IO), with MongoDB for data persistence. Key implemented features include:

- **Account Management:** Users can create an account, authenticate (login/logout), edit their profile, and upload an avatar. Session-based authentication (using secure cookies) manages user sessions.
- **Matching & Courses:** Students are scoped to their university and can add courses to their profile. The system matches students by common courses and availability, showing potential study buddies and overlapping courses.
- **Real-Time Chat & Notifications:** Users can communicate in one-to-one or group chatrooms. Messages are saved to history, and real-time notifications alert users to new messages, users joining/leaving chats, and scheduling updates.
- **Scheduling & Availability:** Users set weekly availability and can propose or join study sessions. Session details (title, time, location, description) are coordinated through the app.
- **Responsive UI & UX:** The interface is built with Bootstrap and a custom theme to ensure usability across desktop, tablet, and mobile devices.
- **Security & Observability:** The application uses modern security practices (bcrypt password hashing, HTTP-only cookies, helmet headers, optional rate limiting) and provides monitoring endpoints (healthz, /readyz, /status) plus Prometheus metrics and structured logs.

This report gives an overview of StudyBuddy, with all the main features, requirements, and design details. It contains architecture diagrams, a project timeline, and notes on risks and mitigation. It also points out the design patterns and quality measures used while building the system.

2 Objectives

The StudyBuddy system is a web application that helps students connect for collaborative studying. Its frontend is built as a React/TypeScript single-page app, while the backend uses Node.js and Express. The frontend talks to the backend through REST APIs and real-time WebSocket events powered by Socket.IO. Data is stored in MongoDB, and user sessions are handled securely using cookies and an express-session store.

- **University Scoping:** Restrict connections so students only match with others at the same institution.
- **Account Management:** Allow users to sign up, log in, and manage their profiles and avatars
- **Course Enrollment:** Enable users to add and remove courses in their profile and see course overlaps with other students.
- **Matching:** Provide functionality to search for and list compatible study partners based on shared courses and availability.
- **Communication:** Support one-to-one and group chat with message history and real-time updates.
- **Scheduling:** Allow students to set weekly availability and propose/accept study session invitations (with time and location).
- **Usability:** Offer a responsive user interface that works on desktop, tablet, and mobile layouts.
- **Security & Observability:** Implement industry-standard security practices (password hashing, secure cookies, helmet headers) and provide monitoring (health checks, metrics, structured logging).

3 Functional Requirements

FR1
Title: Account Management
Priority: High
Description: Users can sign up, log in, and log out; manage profile details and avatar.
Acceptance Criteria: Signup creates a new user; login creates a session; logout ends the session; profile changes (including avatar upload) are persisted.
Implementing Files: backend/routes/users.js, backend/user.model.js, backend/multer-config.js; frontend/src/Components/SignupPage.tsx, LoginPage.tsx, EditProfile.tsx.

FR2
Title: University Scoping
Priority: High
Description: Enforce that users only match with others from the same university.
Acceptance Criteria: Matching and search results return only users who share the same university.
Implementing Files: backend/user.model.js (stores university); matching/search endpoints filter by university.

FR3
Title: Course Management
Priority: High
Description: Users can add or remove courses from their profile and view overlaps.
Acceptance Criteria: Users can add courses via the UI; courses are stored in the profile; overlapping courses with other users are displayed.
Implementing Files: backend/routes/scheduling.js (courses endpoints), backend/scheduling.model.js; frontend/src/Components/EditProfile.tsx, ShowBuddies.tsx.

FR4
Title: Matching
Priority: High
Description: Find and list compatible study buddies based on shared courses and availability.
Acceptance Criteria: Search returns users enrolled in common courses and matching availability; results provided in JSON.
Implementing Files: backend/routes/scheduling.js (/find-partners endpoint), backend/routes/match.js; frontend/src/Components/Matching/MatchUsersList.tsx.

FR5
Title: Real-Time Chat
Priority: High
Description: One-to-one and group chat functionality with message history.
Acceptance Criteria: Users can create/join chatrooms and send/receive messages in real time; chat history is saved.
Implementing Files: backend/routes/chat.js, backend/sockets/chatSocket.js, backend/chatroom.model.js, backend/message.model.js; frontend/src/Components/ChatList/ChatList.tsx, Chatroom.tsx.

FR6
Title: Notifications
Priority: Medium
Description: Real-time notifications for new messages, user joins/leaves, and session updates.
Acceptance Criteria: In-chat notifications appear when others send messages or join; schedule updates trigger notifications.
Implementing Files: backend/sockets/chatSocket.js (emits notification events); frontend/src/Components/Notifications
Notifications.tsx.

FR7
Title: Scheduling
Priority: High
Description: Propose, accept, and manage study sessions; set weekly availability.
Acceptance Criteria: Users can set availability slots; create a session invitation; invited users see pending sessions.
Implementing Files: backend/routes/scheduling.js (/availability, /sessions endpoints), backend/scheduling.model.js; frontend/src/Components/Scheduling/Scheduling.tsx.

FR8
Title: Search/Filter
Priority: Medium
Description: Discover buddies by filtering on course and time availability.
Acceptance Criteria: The matching interface allows filtering by specific course and time; results update accordingly.
Implementing Files: backend/routes/scheduling.js (/find-partners with query parameters); frontend/src/Components/

FR9
Title: Responsive UI
Priority: Medium
Description: Application layout adapts to desktop, tablet, and mobile screen sizes.
Acceptance Criteria: The UI renders properly across different screen widths (as verified through responsive design testing).
Implementing Files: frontend/src/styles/theme.css; Bootstrap classes used in layout components.

The functional requirements listed above define the core capabilities and behaviors expected from the StudyBuddy system. Each requirement ensures that the application meets both user needs and technical standards, covering essential aspects such as account management, university-based scoping, course tracking, matching algorithms, real-time communication, scheduling, notifications, and responsive design. By specifying clear acceptance criteria and the implementing files for each functionality, this section provides traceability from requirements to code implementation. Together, these functional requirements serve as a foundation for designing, developing, and testing the system, ensuring a reliable and user-friendly experience for students collaborating in study sessions.

4 Non-Functional Requirements

The StudyBuddy platform is built to deliver not just core functionality, but also high standards in speed, safety, ease of maintenance, monitoring, and accessibility. This section details each non-functional requirement area, reflecting the latest improvements in the system.

4.1 Performance

StudyBuddy is optimized for fast responses and low latency, even under heavy usage. The main focus is to reduce backend load, lower server CPU usage, and make the user experience smooth and interactive. Optimizations include debouncing search and pagination requests on the frontend, using lean queries, adding database indexes, and caching frequent responses in-memory. Regular benchmarking tracks improvements with objective metrics.

- Debouncing search and pagination requests lowers unnecessary backend calls.
- Lean MongoDB queries and effective index management reduce data fetch times.
- Aggregation facets enable quick page counts and listing, improving batch data speed.
- Redis-based hot caching and backend compression minimize wait times and network usage.
- Benchmark logs monitor p50, p95, and p99 latency for reproducible performance evidence.

Metrics Summary

Generated at: 2025-11-12T13:56:26.274Z

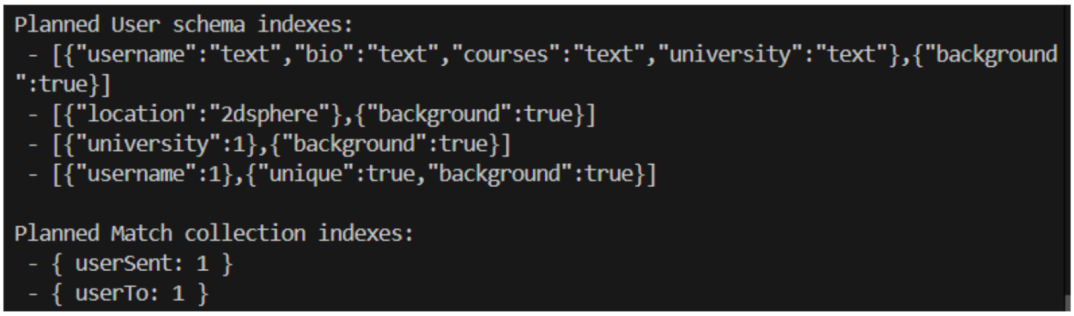
Perf logs (per-route)

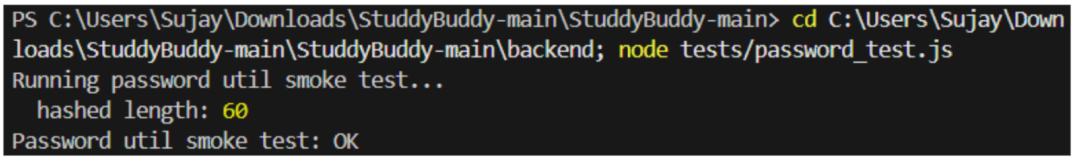
Route	count	mean (ms)	p50	p95	p99	min	max
/check-logged-in	64	691.66	7.335000000000001	2458.6219999999994	3393.9857999999954	3.52	4487.25
/get-users	62	519.42	7.1899999999999995	1966.8235	2388.0266	2.34	2612.47
/sessions	6	412.43	8.765	1406.75	1554.71	6.47	1591.7
/image/:username	30	659.64	10.46	2813.4214999999986	3338.2648000000004	3.72	3479.46
/availability	6	517.96	10.695	1817.9900000000002	2042.9660000000003	5.86	2099.21
/buddies	355	257.12	8.94	1351.4990000000007	2286.451199999992	1.76	3987.74
/info	10	254.06	15.85	1229.3719999999992	1503.5624	12.22	1572.11
/candidates	625	335.98	10.8	1610.4479999999999	3781.2959999999966	1.85	6325.02
/	266	335.43	6.155	1782.9825	2736.7760000000035	2.77	4497.58
/logout	1	820.39	820.39	820.39	820.39	820.39	820.39
/auth	1	94.28	94.28	94.28	94.28	94.28	94.28
/:id	4	47.85	9.785	140.28099999999995	158.67219999999998	8.57	163.27
/:id/users	4	198.85	155.39	329.1529999999999	352.4545999999999	126.33	358.28
/addsinglebuddy	1	57.18	57.18	57.18	57.18	57.18	57.18
/matchedbuddyinfo	6	9.31	8.725	12.58	13.436	7.38	13.65

4.2 Security

Robust security practices guard both data and user activity at all levels. Authentication, session handling, secure transport headers, and runtime monitoring have been reinforced. Every control is visible in configuration and log evidence.

- Sessions use httpOnly, sameSite, and secure flags, stored in MongoDB.
- HTTP headers are protected and compressed using helmet.
- Passwords are hashed and salted with bcrypt; session regeneration prevents fixation.
- Rate limiting, backed by Redis, avoids abuse and adapts to scale.
- Prometheus metrics and structured logs let developers audit all protections.

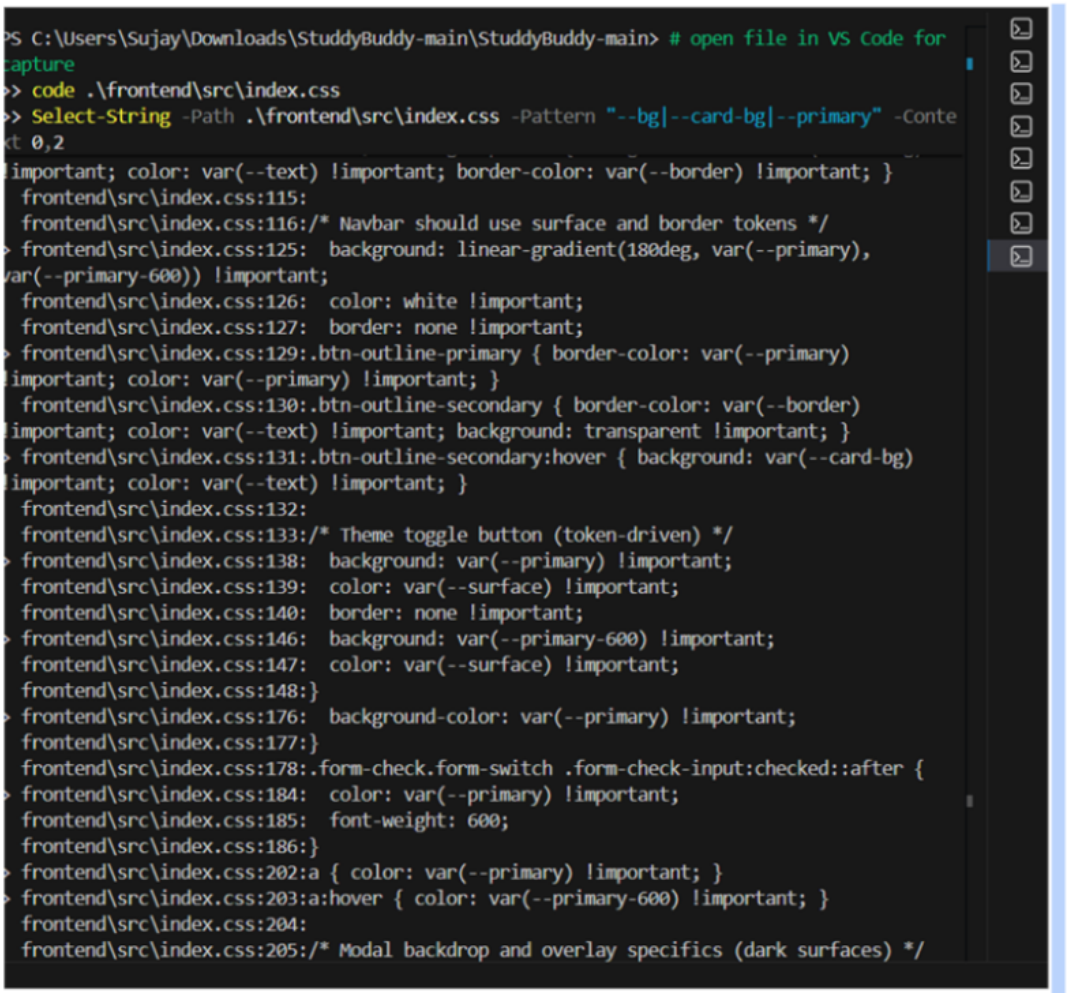




4.3 Maintainability

StudyBuddy is designed for easy extension, troubleshooting, and developer onboarding. Code modules and configuration files are organized to minimize complexity and make future changes safer.

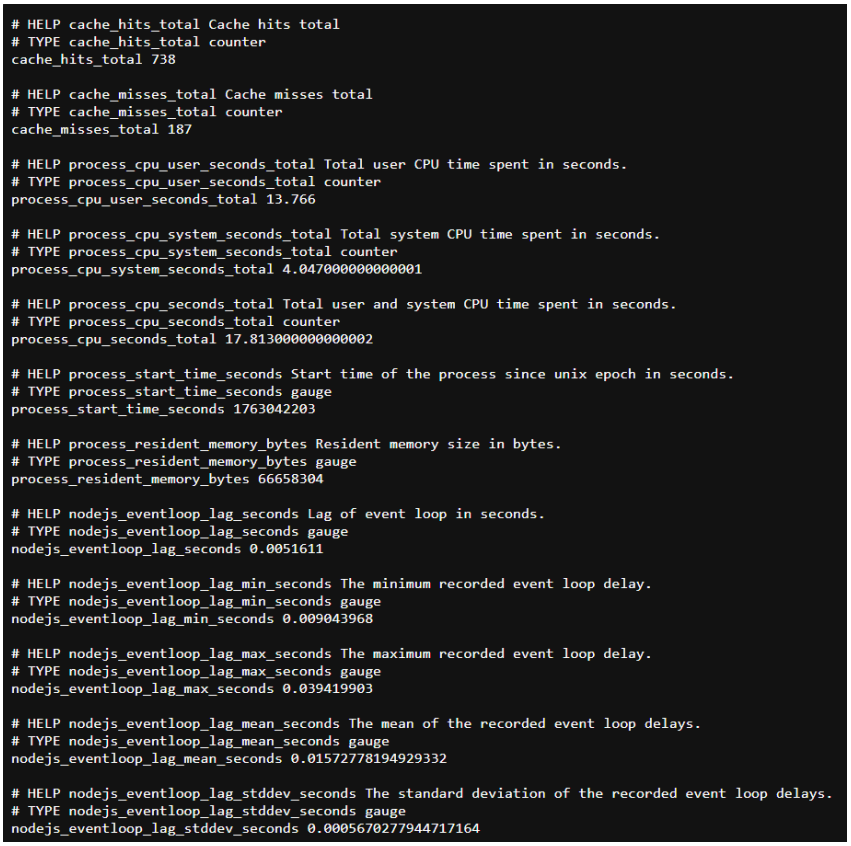
- Folders are separated by function, with clear boundaries for routes, models, middlewares, and components.
- Configuration is centralized using .env files and environment variables.
- Key environment settings are documented for quick setup.
- Request IDs and performance logs help new and returning developers quickly trace issues.



4.4 Observability

The platform provides strong live monitoring and debugging tools. Health check endpoints, structured logging, and Prometheus-compatible metrics ensure that system status and past activity are always accessible.

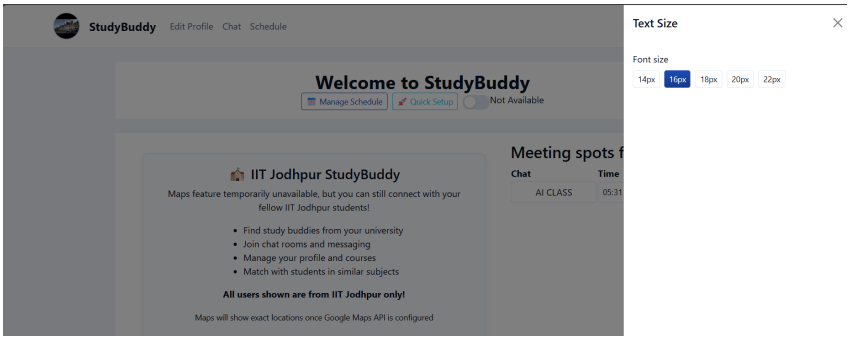
- Health, readiness, and status endpoints provide live feedback for monitoring.
- All logs use request IDs and NDJSON format for easy traceability.
- Metrics endpoints expose system properties for Prometheus dashboards.
- Historical performance logs make triage and root-cause analysis easier.



4.5 Accessibility and Usability

User experience upgrades make StudyBuddy accessible for all, including those with vision or dexterity limitations. Controls for visual themes and font size, consistent contrast, fallback images, and keyboard-friendly navigation create an inclusive product. Changes are validated both automatically and manually.

- UI theme toggles and font size controls give users direct personalization.
- Semantic color tokens ensure consistent contrast throughout the interface.
- Fallback avatars with alt text aid users when images fail to load.
- All key controls support keyboard navigation and ARIA labels for screen reader support.
- Accessibility is checked using automated tools and manual steps, with results documented for review.



Key Points

- Each system attribute has been technically strengthened with best-practice solutions.
- Guides, logs, and measurable metrics provide full transparency and reproducibility for all reviewers.
- Monitoring, performance, and compliance tools are integrated to make ongoing maintenance simple.
- All improvements are recorded in project files and supported by clear evidence from code and runtime outputs.

5 Architecture

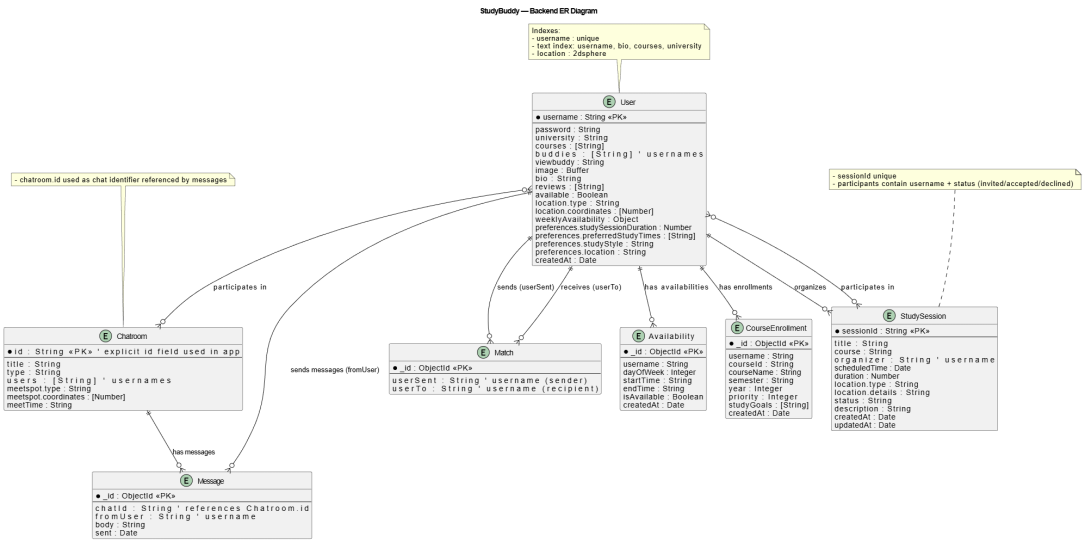


Figure 1: StudyBuddy — Backend ER Diagram

The StudyBuddy system is built on a Client–Server Single-Page Application (SPA) model. The frontend is a React application that runs in the user's browser and communicates with the backend, which is implemented using Node.js and Express. The system supports both HTTP REST API requests and real-time communication through Socket.IO to enable interactive features. The backend follows a layered architecture: routers and controllers manage incoming requests, business logic is handled in services and models, and Mongoose models (following the Active Record pattern) handle database operations. Common middleware such as helmet, CORS, session management, rate limiting, logging, and metrics are applied across all requests.

User authentication is session-based. When a user logs in, an express-session cookie is set with httpOnly, sameSite=lax, and secure flags and is stored in MongoDB using connect-mongo. Passwords are stored securely with bcrypt, and legacy plaintext passwords are upgraded automatically upon login. Additional security measures include HTTP header protection through helmet and optional login rate limiting.

The system's observability features include /healthz, /readyz, and /status endpoints that provide service status, as well as a /metrics endpoint for Prometheus monitoring. Logging is handled with Winston in a structured format, with request IDs included for easier tracing. Real-time functionalities, such as chat and study session scheduling, use dedicated Socket.IO namespaces (/chat and /meet-up) to broadcast events and notifications.

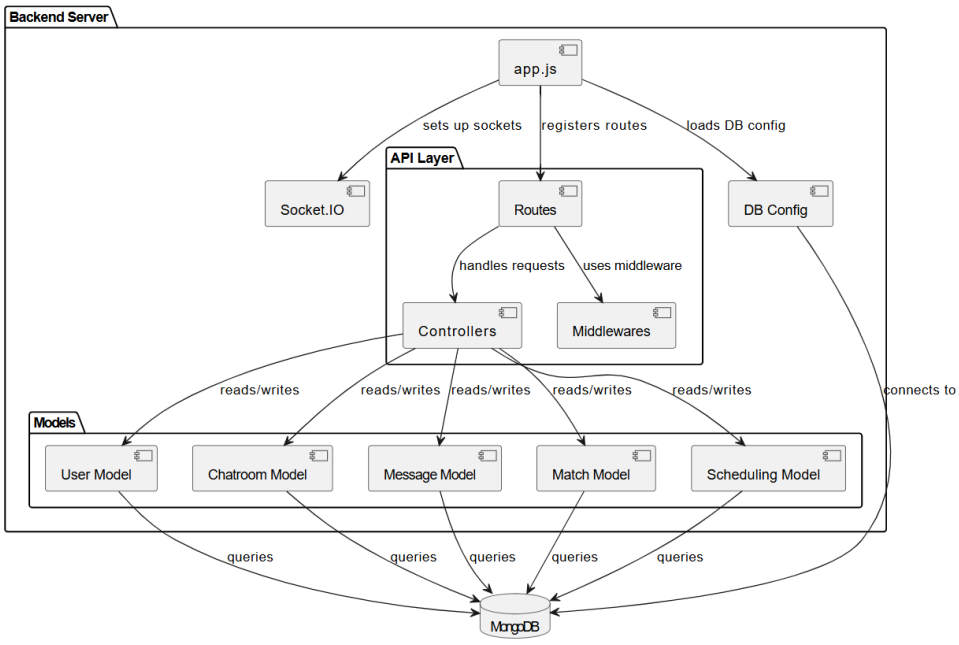


Figure 2: This diagram explains backend internals. app.js initializes the server, registers routes, configures middleware, and sets up Socket.IO for real-time messaging. Routes forward requests to controllers, which apply business logic and coordinate with models for users, chatrooms, messages, matches, and scheduling. Each model connects to MongoDB, ensuring consistent data management across the application.

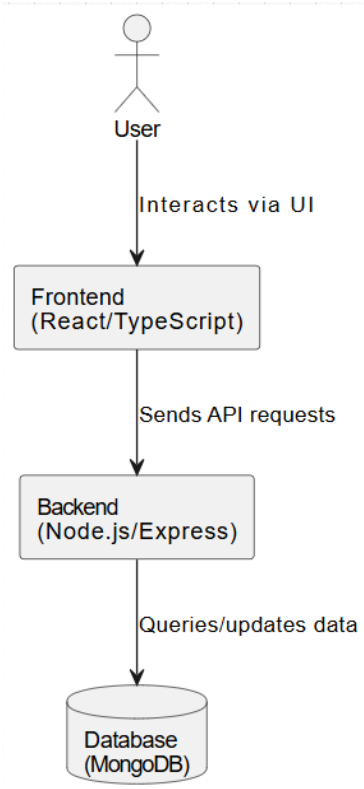


Figure 3: This diagram presents Studybuddy's system at a glance. The user interacts through the React/TypeScript frontend, which provides the interface and sends API calls or WebSocket messages to the Node.js/Express backend. The backend processes logic, manages sessions, and communicates with MongoDB for persistent storage of users, messages, matches, and schedules.

Figure 4: Explains request processing flow in Express. Incoming requests pass through middleware for authentication, validation, and logging before reaching controllers. Ensures security, modularity, and systematic handling of user requests.

Figure 5: Shows React frontend routes mapping. Defines navigation paths for different features (dashboard, authentication, chat, scheduling, matching), providing a structured single-page application experience for Studybuddy users.

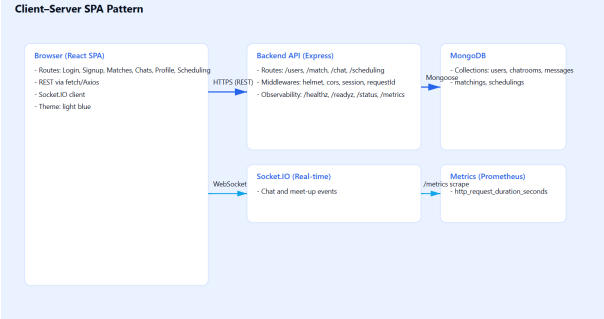


Figure 6: Demonstrates client-server interaction in a single-page app. React frontend requests data from Express backend, which queries MongoDB. Backend responds with JSON, enabling dynamic UI updates without page reloads.

Figure 7: Outlines user authentication process: signup/login requests pass through middleware, controllers validate credentials, backend queries database, and successful responses return tokens, granting secure access to frontend features.

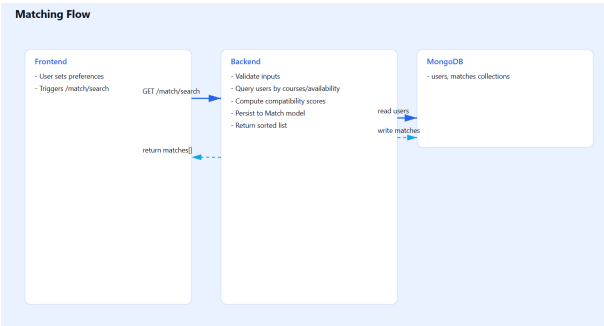


Figure 8: Shows matching logic. User preferences are compared, backend algorithms identify best matches, and results are returned to frontend. Facilitates intelligent peer connections within the Studybuddy platform.

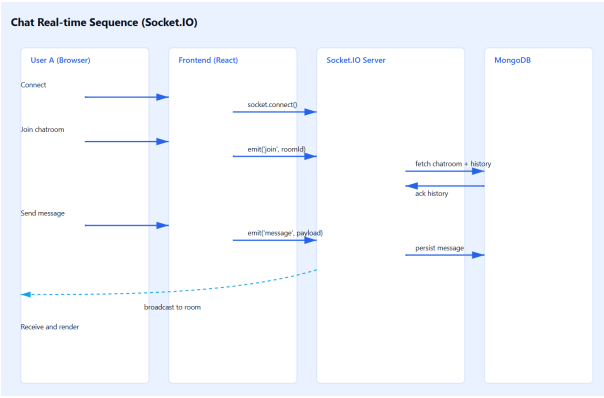


Figure 9: Depicts real-time chat sequence. Messages sent by a user travel through Socket.IO backend, are stored in MongoDB, and broadcast instantly to other connected clients.

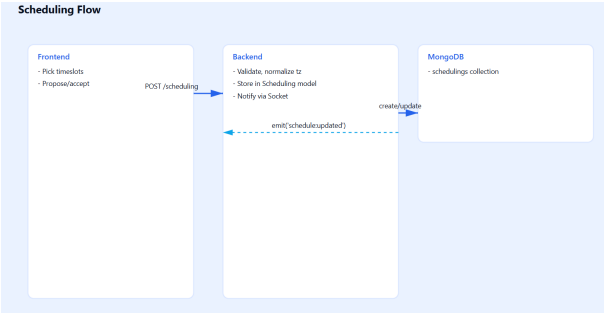


Figure 10: Illustrates scheduling process. User selects time, backend validates conflicts, stores event in database, and updates frontend. Ensures seamless booking and management of study sessions.

6 Design Patterns and Quality Attributes

- The backend is organized into layers—routes (presentation), controllers (business logic), and models (data access)—which separates concerns and simplifies debugging and maintenance.
- While the backend uses Express with layered logic, the overall system follows an MVC-like pattern where React (View) interacts with controllers (via APIs) and models (via Mongoose).
- Implemented using Socket.IO, this ensures clients are notified in real time when events occur (e.g., new chat messages, study session updates).
- MongoDB connections are created as singletons to avoid redundant connections, and middleware follows reusable factory-like design to enforce validation and security.
- The system emphasizes security (bcrypt password hashing, cookies with secure flags), maintainability (modular structure), usability (responsive UI), and observability (metrics and logs).

7 Conclusion

The StudyBuddy platform demonstrates a comprehensive integration of modern software engineering principles across its full-stack architecture. This project has implemented robust mechanisms for user authentication, data privacy, and session management, leveraging secure cookies and bcrypt hashing to protect sensitive information. Performance enhancements, such as client-side debouncing, lean database queries, effective indexing, and Redis-based hot caching, allow the system to deliver responsive user experiences, even under high load scenarios. Observability is ensured by structured logging, health check endpoints, and Prometheus-compatible metrics, making it easier to monitor, diagnose, and maintain the application in production. The codebase follows a modular structure with centralized configuration and extensive developer documentation, supporting rapid onboarding and safe extension. Accessibility upgrades, including theme toggles, adjustable font sizes, semantic colors, and keyboard-friendly controls, make the application inclusive for all users. Together, these advancements result in a scalable, secure, maintainable, and user-centric platform ready for future enhancements and feature expansions. The technical choices and design patterns adopted lay a strong foundation for continued reliability and operational excellence.